Session 3.3: Practical Machine Learning with Scikit-Learn: Model Training and Validation

TODAY'S PURPOSE

Why This Session Matters

- Train your first models with Scikit-Learn
- Make predictions on new cases
- Measure how accurate the predictions are

Today's flow: 60 min lecture, then 120 min hands-on lab

EXPECTED OUTCOMES

By the end of today, you will be able to:

1. Train linear and logistic regression models
2. Evaluate with Root Mean Squared Error (RMSE) and cross-validation
3. Predict one case and fix a 1D array ValueError

Concepts for This Session

CONCEPT 1

Unified Estimator API

The standard three-stage Scikit-Learn interface: instantiate model, fit on training data (.fit()), predict on new data (.predict()), and evaluate accuracy (.score()).

CONCEPT 2

Linear Regression

A fundamental supervised regression algorithm that models the linear relationship between continuous independent variables and a target business metric.

CONCEPT 3

Logistic Regression

A foundational supervised classification algorithm that models the probability of a binary categorical event using the logistic sigmoid function.

CONCEPT 4

K-Fold Cross-Validation

Splitting training data into K subsets to iteratively train and validate the model K times, providing an unbiased estimate of generalization performance.

How the Concept Works in Practice

FOUNDATION 1

The Scikit-Learn Estimator Interface

Every model in Scikit-Learn (whether simple linear regression or advanced random forests) implements the identical class interface. Learning .fit() and .predict() unlocks hundreds of advanced machine learning algorithms.

FOUNDATION 2

Linear Regression for Continuous Business Forecasting

Quantifying empirical business relationships. Model coefficients (model.coef_) indicate the marginal financial return of adding square footage or reducing distance to commercial business districts.

FOUNDATION 3

Logistic Regression for Binary Operational Decisions

Unlike linear regression which can predict values outside [0, 1], logistic regression squashes predictions into calibrated probabilities between 0.0 and 1.0, ideal for loan default or churn risk scoring.

FOUNDATION 4

Overfitting, Underfitting, and Generalization

A model that achieves 100% accuracy on training data but fails on real-world test data has memorized noise (overfitting). Cross-validation ensures models generalize reliably to future enterprise data.

Build, Test, and Diagnose in the IDE

[1] FILE EXPLORER

Organize Assets

Use the session dataset and keep scripts in a clear workspace.

[2] CODE EDITOR

Write Small Segments

Run each completed segment before expanding the program.

[3] AI AGENT CHAT

Diagnose Errors

Capture the traceback, request an explanation, and review the fix.

[4] INTEGRATED TERMINAL

Verify Execution

Confirm the final program completes with return code 0.

Script Demonstration 1: Practical Machine Learning with Scikit-Learn: Model Training and Validation

```
import pandas as pd
    from sklearn.linear_model import LinearRegression
    from sklearn.model_selection import train_test_split

    df = pd.read_csv("data/session_03_03_dataset.csv")
    X = df[["Square_Feet_Units", "Bedrooms_Count",
"Distance_To_CBD_Miles", "Local_Tax_Rate_Pct"]]
    y = df["Sale_Price_USD"]

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.20, random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)
    print("Model Intercept:", round(model.intercept_, 2))
    print("Model Coefficients (Feature Weights):", dict(zip(X.columns,
model.coef_.round(2))))
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 2: Practical Machine Learning with Scikit-Learn: Model Training and Validation

```
import numpy as np
    from sklearn.metrics import mean_squared_error, r2_score

    y_pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    print(f"Test Set RMSE: ${rmse:,.2f}")
    print(f"Test Set R-Squared (Variance Explained): {r2:.4f}")
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 3: Practical Machine Learning with Scikit-Learn: Model Training and Validation

```
from sklearn.model_selection import cross_val_score

    cv_scores = cross_val_score(model, X, y, cv=5, scoring="r2")
    print("5-Fold Cross-Validation R-Squared Scores:",
cv_scores.round(3))
    print(f"Mean Cross-Validated R-Squared: {cv_scores.mean():.3f}
(+/- {cv_scores.std():.3f})")
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED

Foundational Practice

Follow the lab protocol, run the script, and verify its output with the session dataset.

EXERCISE 2: DEBUGGING

Applied Challenge

Extend the program, diagnose an intentional error, apply the verified correction, and confirm return code 0.